

1) Assembly Programming

2) Process Management

Signals {
 → InterProcess Communication
 → Process Synchronization

3) Memory Management

→ Paging & Virtual Memory.

→ Segmentation

4) Device & File Management

→ Virtual File System

→ File System Structure (inodes, etc.)

→ Handling file permissions

→ Disk I/O Scheduling,
· Linux elevator.

Pipes, FIFOs, sockets, message queues

Signals → Allow kernel to communicate
with a process.

Kernel needs to notify a process
asynchronously.

Three types of I/O

→ 1) synchronous

2) synchronized → O_SYNC flag

3) asynchronous $\rightarrow$ Returns immediately without completing the operation, but ~~at~~ on completion of the operation, notifies the process by sending a signal.

When the operation is complete, the kernel is notified, a signal gets registered and then the kernel sends the signal to the process. Each process has its own signal handler. On receiving a signal, the normal sequence of instruction execution is interrupted and execution ~~at~~ jumps to

the signal handler.

SIGKILL, SIGABRT, SIGINT

```
static void sigint_handler (int signo) {  
    printf ("SIGINT raised");  
}
```

```
}
```

```
if (signal (SIGINT, sigint_handler)  
    == SIG_ERR) {  
    /* Error handling */  
}
```

```
}
```

```
kill (pid_t pid, int signo);
```

Signals are an interprocess communication mechanism

Signals are also a mechanism for the kernel to indicate errors to a process.

Signals create the problem of race conditions. Blocking signals is possible. If blocking is essential, then we say that the code is not reentrant; otherwise it is reentrant. POSIX specifies a list of reentrant functions, which do not require blocking of signals.

What if the code is not reentrant?
→ Signals need to be blocked/masked.

`sigprocmask` (int how, signal *new, signal *old);
how = { SET_MASK,
SET_BLOCK,
SET_UNBLOCK }

how = SET_MASK

new = ~~new~~ {SIGINT}

old = {SIGKILL}

Both {SIGINT, SIGKILL} are blocked.

If how = SET_BLOCK,
new = {SIGINT}

old = {SIGKILL},

ONLY {SIGINT} gets blocked.

Suppose the main memory/RAM has a mean access time of 50 ns. There is a two-level page table, with a TLB hit rate of 0.95, and access time of 5 ns. The cache latency is of 2 ns and has a hit rate of 0.9. What is the total time (mean) required

to access data by the processor?

To access the physical memory: $\rightarrow$

(i) Case 1 $\rightarrow$ TLB hit

$\rightarrow 5 \text{ ns}$

(ii) Case 2 $\rightarrow$ TLB miss

$$50 \times 2 + 5 = 105 \text{ ns}$$

Average ~~acc~~ time to access the page

$$\text{table} = 5 \times 0.95 + 105 \times 0.05$$

$$= ~~100 \times~~ 5 + 100 \times 0.95$$

$$= ~~100 \text{ ns}~~$$

$$10 \text{ ~~ns~~ ns}$$

(A)

(i) Cache Hit $\rightarrow 2 \text{ ns}$ (90%)

Cache miss $\rightarrow 50 \text{ ~~ns~~ ns}$ (10%)
(50 + 2)

Total access time =

$$\begin{aligned} & \underset{(A)}{10} + 2 \times 0.9 + 52 \times 0.1 \\ &= 10 + 1.8 + 5.2 \\ &= 17 \text{ ns} \end{aligned}$$

On a disk, suppose you have an average seek latency of 10 ms per track and there are 50 tracks. There are also 100 sectors in each track, and a rotational latency of 1 ms per sector. A single sector takes another 20 ms to transfer the data. What is the total latency to access the disk if requests are random?

$$\text{Disk latency} = \text{Seek latency} + \text{Rotational latency} + \text{Transfer latency}$$

$$\begin{aligned} &= 250 + 50 + 20 \\ &= 320 \text{ ms} \end{aligned}$$